

 NVR Fabric CI/CD Pipeline - Complete Overview

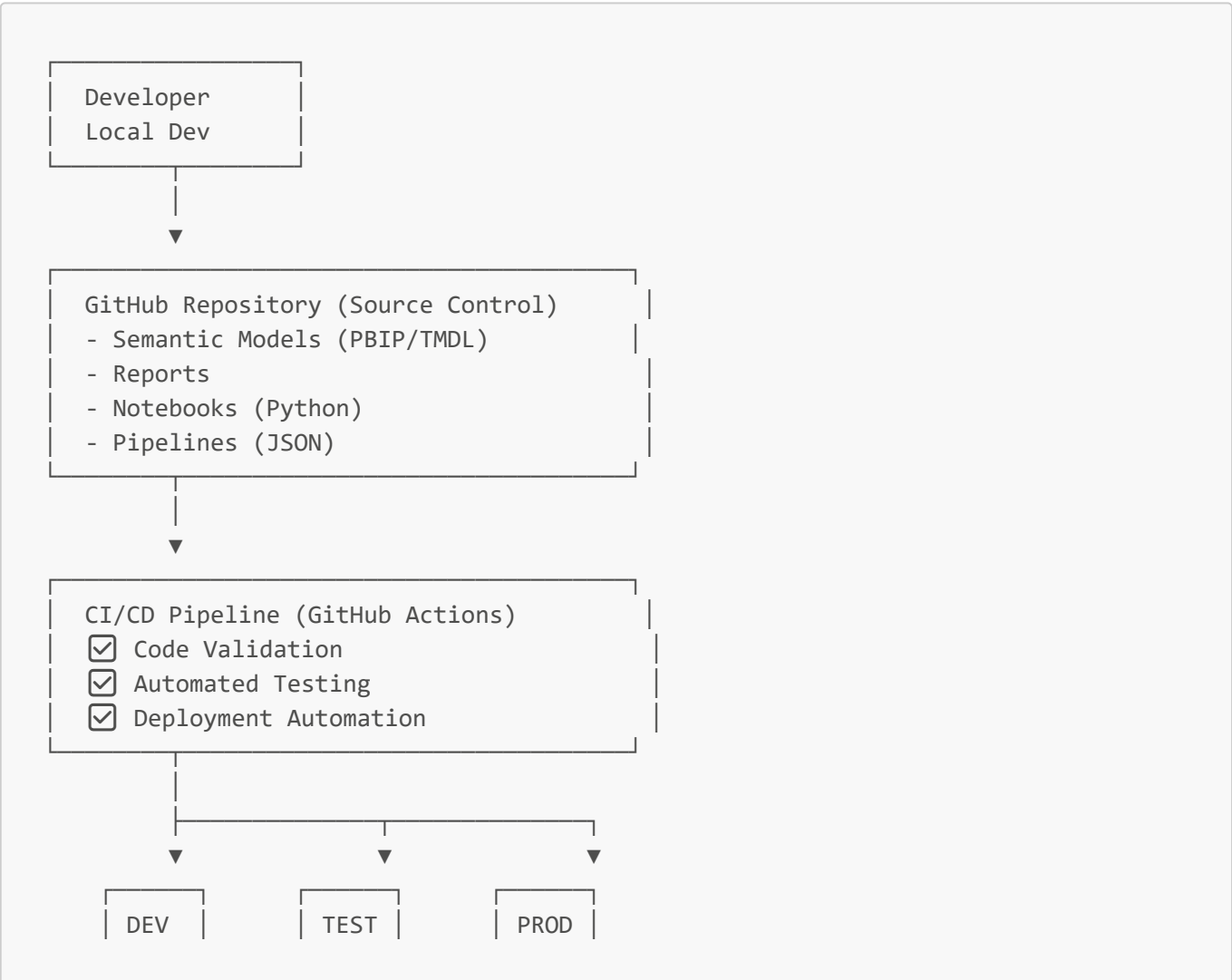
Executive Summary

This document outlines the complete CI/CD (Continuous Integration/Continuous Deployment) pipeline for the NVR Data Science team's Microsoft Fabric workspace, enabling automated deployment, testing, and quality assurance.

 Table of Contents

- 1. [Pipeline Architecture](#)
- 2. [Workflow Overview](#)
- 3. [Environment Strategy](#)
- 4. [Deployment Process](#)
- 5. [Quality Gates](#)
- 6. [Rollback Strategy](#)
- 7. [Getting Started](#)

 Pipeline Architecture



Env

Env

Env

Workflow Overview

1. PR Validation Workflow

File: `.github/workflows/pr-validation.yml`

Triggers:

- Pull request to `main` or `develop` branches

Steps:

1. ☒ **Code Quality Checks**
 - Black formatting validation
 - Flake8 linting
2. ☒ **Artifact Validation**
 - Notebook syntax validation
 - Pipeline JSON schema validation
 - Configuration file validation
3. ☒ **Security Scanning**
 - Detect hardcoded secrets
 - Check for sensitive data exposure
4. ☒ **Unit Testing**
 - Run Python unit tests
 - Validate data quality checks

Result: PR can only be merged if all checks pass ☒

2. Deploy to DEV Workflow

File: `.github/workflows/deploy-dev.yml`

Triggers:

- Push to `main` branch
- Changes in `notebooks/`, `pipelines/`, or `config/` directories
- Manual trigger via GitHub Actions UI

Steps:

1.  **Authentication**

- Azure AD authentication
- Acquire Fabric API token

2. **Change Detection**

- Identify modified notebooks
- Identify modified pipelines
- Identify modified semantic models

3. **Deployment**

- Deploy notebooks to Fabric workspace
- Deploy data pipelines
- Update semantic models

4. **Validation**

- Verify deployments
- Run smoke tests
- Generate deployment report

Target: NVR-Dev Fabric workspace

3. Deploy to TEST Workflow

File: [.github/workflows/deploy-test.yml](#)

Triggers:

- Manual approval after successful DEV deployment
- Tag creation ([release/*](#))

Additional Steps:

-  **Integration Testing**
 - End-to-end pipeline execution
 - Data quality validation
 - Model training validation
-  **Performance Testing**
 - Query performance benchmarks
 - Resource utilization checks

Target: NVR-Test Fabric workspace

4. Deploy to PROD Workflow

File: [.github/workflows/deploy-prod.yml](#)

Triggers:

- Manual approval required (protected environment)
- Requires successful TEST deployment

Safety Features:

-  **Automatic Backup** before deployment
-  **Rollback Plan** prepared
-  **Notifications** to stakeholders
-  **Approval Required** from designated reviewers

Target: NVR-Production Fabric workspace

5. Data Quality Check Workflow

File: [.github/workflows/data-quality-check.yml](#)

Triggers:

- Scheduled (daily at 6 AM UTC)
- Manual trigger

Purpose:

- Validate data integrity in lakehouses
- Check for data drift
- Alert on anomalies
- Generate quality reports

6. Scheduled Model Training Workflow

File: [.github/workflows/scheduled-model-training.yml](#)

Triggers:

- Scheduled (weekly on Mondays at 8 AM UTC)
- Manual trigger

Purpose:

- Execute model training notebooks
- Log metrics to MLflow
- Compare model performance
- Update production models if improved

Environment Strategy

Environment	Purpose	Protection	Approval Required
-------------	---------	------------	-------------------

Environment	Purpose	Protection	Approval Required
DEV	Development & testing	None	 No
TEST	Integration testing & UAT	Basic	 Recommended
PROD	Production workloads	Strict	 Required

Environment Configuration

Each environment has its own configuration file:

```
config/
├── dev.yml      # Development settings
├── test.yml     # Test environment settings
└── prod.yml    # Production settings
```

Configuration includes:

- Workspace names and IDs
- Deployment settings
- Notification channels
- Feature flags
- Resource limits

Deployment Process

Step-by-Step Deployment Flow

1. Developer Creates Feature Branch

```
git checkout -b feature/new-ml-model
# Make changes to notebooks, models, etc.
git add .
git commit -m "Add new churn prediction model"
git push origin feature/new-ml-model
```

2. Create Pull Request

- PR automatically triggers validation workflow
- Automated checks run (code quality, tests, validation)
- Team reviews code changes
- Checks must pass before merge 

3. Merge to Main

- Triggers automatic deployment to DEV

- Changes deployed to NVR-Dev workspace
- Smoke tests run automatically
- Deployment notification sent

4. **Promote to TEST** (Manual)

- Navigate to GitHub Actions
- Select "Deploy to Test"
- Click "Run workflow"
- Monitor deployment progress
- Integration tests run automatically

5. **Promote to PROD** (Manual + Approval)

- Navigate to GitHub Actions
- Select "Deploy to Prod"
- Request approval from designated reviewers
- Wait for approval ⌚
- Deployment executes with backup
- Production validation runs

Quality Gates

Pre-Merge Gates (PR Validation)

☒ **Must Pass:**

- ☐ Python code formatting (Black)
- ☐ Linting (Flake8)
- ☐ Notebook validation
- ☐ Pipeline schema validation
- ☐ Unit tests (100% pass rate)
- ☐ No hardcoded secrets
- ☐ Code review approved

Post-Deployment Gates

☒ **DEV Deployment:**

- ☐ Artifacts successfully deployed
- ☐ Smoke tests pass
- ☐ Workspace accessible

☒ **TEST Deployment:**

- ☐ All DEV gates passed
- ☐ Integration tests pass
- ☐ Performance benchmarks met
- ☐ Data quality checks pass

☑ **PROD Deployment:**

- ☐ All TEST gates passed
 - ☐ Backup completed successfully
 - ☐ Stakeholder approval obtained
 - ☐ Rollback plan ready
 - ☐ Production validation pass
-

← **Rollback Strategy**

Automatic Rollback Triggers

The pipeline automatically rolls back if:

- Deployment fails ✖
- Smoke tests fail ✖
- Critical errors detected ⚠

Manual Rollback Process

```
# Script available: scripts/rollback_deployment.py
python scripts/rollback_deployment.py \
  --workspace-id <WORKSPACE_ID> \
  --backup-id <BACKUP_ID> \
  --environment prod
```

Backup Strategy

- ☒ Automatic backups before PROD deployments
 - ☒ Backups stored in Azure Blob Storage
 - ☒ 30-day retention period
 - ☒ Point-in-time recovery capability
-

🌀 **Getting Started**

Prerequisites

1. **GitHub Access**

- Repository access with write permissions
- Ability to create branches and PRs

2. **Azure Credentials**

- Azure AD authentication configured
- Service Principal with Fabric permissions

3. **Required Secrets** (GitHub Repository Settings)

```
AZURE_CLIENT_ID      # Service Principal Client ID
AZURE_TENANT_ID       # Azure AD Tenant ID
AZURE_SUBSCRIPTION_ID # Azure Subscription ID
FABRIC_DEV_WORKSPACE_ID # Development Workspace ID
FABRIC_TEST_WORKSPACE_ID # Test Workspace ID
FABRIC_PROD_WORKSPACE_ID # Production Workspace ID
```

Making Your First Deployment

1. Clone the repository

```
git clone https://github.com/sautalwar/nvrfabricdemo1.git
cd fabric-cicd-demo
```

2. Create a feature branch

```
git checkout -b feature/my-first-change
```

3. Make your changes

- Edit notebooks in `notebooks/`
- Update models in `src/`
- Modify pipelines in `pipelines/`

4. Commit and push

```
git add .
git commit -m "Description of changes"
git push origin feature/my-first-change
```

5. Create Pull Request

- Go to GitHub repository
- Click "New Pull Request"
- Select your branch
- Wait for validation checks ☒
- Request review from team

6. Merge and Deploy

- Once approved, merge PR
- Automatic deployment to DEV starts
- Monitor progress in GitHub Actions tab

Monitoring & Notifications

Deployment Status

Check deployment status at:

- **GitHub Actions Tab:** <https://github.com/sautalwar/nvrfabricdemo1/actions>
- **Email Notifications:** Sent on deployment success/failure
- **Slack Notifications:** Real-time updates in #nvr-deployments channel

Deployment Logs

All deployment logs are available in:

- GitHub Actions workflow runs
- Azure Application Insights
- Fabric workspace activity logs

Support & Troubleshooting

Common Issues

Issue: Deployment fails with authentication error **Solution:** Check Azure credentials and service principal permissions

Issue: Notebook validation fails **Solution:** Run `python scripts/validate_notebooks.py` locally to identify issues

Issue: Pipeline JSON invalid **Solution:** Use online JSON validator or `python scripts/validate_pipelines.py`

Contact

For pipeline issues or questions:

- **GitHub Issues:** Create an issue in the repository
- **Team Lead:** [Contact Info]
- **DevOps Support:** [Contact Info]

Additional Resources

- [Microsoft Fabric Documentation](#)
- [GitHub Actions Documentation](#)
- [Azure DevOps Best Practices](#)
- [Team Confluence Page](#) (Internal)

Last Updated: January 4, 2026

Version: 1.0

Owner: NVR Data Science Team

